



中山大學
SUN YAT-SEN UNIVERSITY

计算机学院（软件学院）
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

Compilation Principle 编译原理

第10讲：语法分析(5)

张献伟

xianweiz.github.io

DCS290, 4/2/2024



中山大學
SUN YAT-SEN UNIVERSITY



Grading[考核标准]

• 编译原理

- 课堂参与（15%）- 点名、提问、测试
- 课程作业（25%）- 5次左右，理论
- 期末考试（60%）- 闭卷

• 编译器构造实验

- 课堂参与（10%）- 签到、练习等
- Project 1（20%）- Lexical Analysis
- Project 2（20%）- Syntax/Semantic Analysis
- Project 3（20%）- IR Generation
- Project 4（30%）- Code Optimization

!!! proj1昨晚已经截止，我们会尽快完成批改！未提交的同学：如果截止前填写了问卷，自动宽限5天，请于4.2/周二23:59:59前提交（宽限期内扣除得分的5%），宽限期外每延一天再扣除得分的10%；没有填写问卷的同学，从今天开始每天扣除10%。proj2/3/4将类似执行，具体有所调整。

• 理论

- 随机点名
 - 缺席优先
- 随机提问
 - 后排优先
- 随机测试
 - 不定时间

• 实验

- 个人完成
 - 杜绝抄袭
- 按时提交
 - 硬性截止
- 侧重代码实现
 - 简略报告

Review Questions

- Q1: actions in top-down parsing?

Expand on non-terminal, compare on terminal.

$S \rightarrow Aa$

$A \rightarrow BD$

$B \rightarrow b$

$B \rightarrow \epsilon$

$D \rightarrow d$

$D \rightarrow \epsilon$

- Q2: how to build the LL(1) parse table?

Two sets: FIRST, FOLLOW

- Q3: for the grammar, what is FIRST(A) and FOLLOW(B)?

$\text{FIRST}(A) = \{b, d, \epsilon\}$, $\text{FO}(A) = \{a\}$ $\text{FOLLOW}(B) = \{d, a\} = \text{FI}(D) + \text{FO}(A)$

- Q4: which one is typically used, LL(0), LL(1), LL(2) ...? Why not others?

LL(1). LL(0) is too weak, LL(k) has a too large table.

- Q5: top-down vs. bottom-up parsing?

Top-down is based on leftmost derivation;

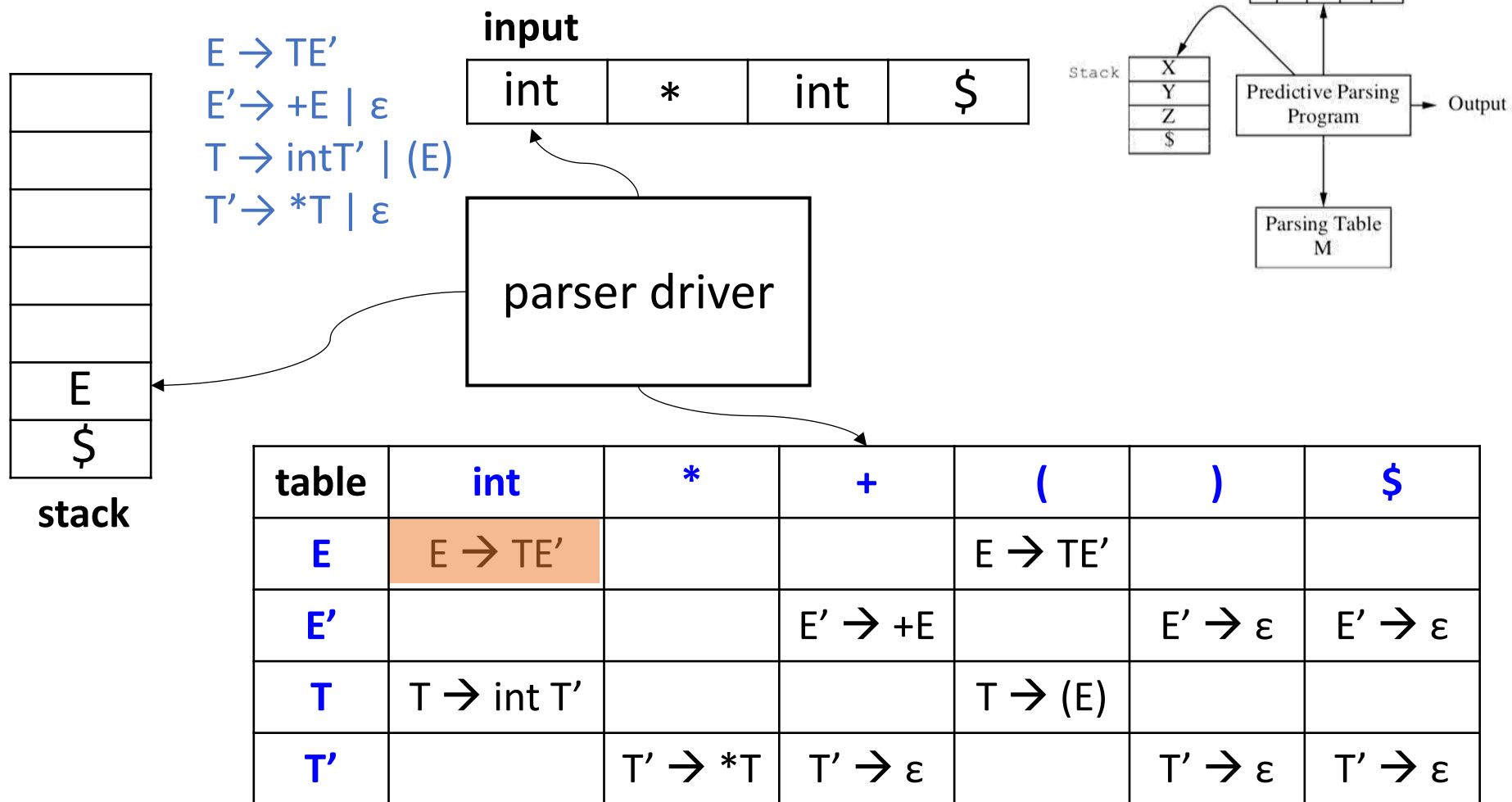
bottom-up is the reverse of rightmost derivation.

Summary: Predictive Parser[小结]

- **FIRST** and **FOLLOW** sets are used to construct **predictive parsing tables**
- Intuitively, **FIRST** and **FOLLOW** sets guide the choice of rules
 - For non-terminal A and lookahead t , use the production rule $A \rightarrow \alpha$ where $t \in \text{FIRST}(\alpha)$
OR
 - For non-terminal A and lookahead t , use the production rule $A \rightarrow \alpha$ where $\epsilon \in \text{FIRST}(\alpha)$ and $t \in \text{FOLLOW}(A)$
 - There can only be ONE such rule
 - Otherwise, the grammar is not LL(1)

LL(1) Parser

- To recognize “int * int”



LL(1) Parse Table

$A \rightarrow \alpha$ (RHS)	FIRST
$E \rightarrow TE'$	int, (
$E' \rightarrow +E$	+
$T \rightarrow intT'$	int
$T \rightarrow (E)$	(
$T' \rightarrow *T$	*
$E' \rightarrow \epsilon$	FOLLOW
$T' \rightarrow \epsilon$	FOLLOW

Symbol	FIRST	FOLLOW
E	int, (	), \$
E'	+, ϵ	), \$
T	int, (	+,), \$
T'	*, ϵ	+,), \$

$E \rightarrow TE'$
 $E' \rightarrow +E | \epsilon$
 $T \rightarrow intT' | (E)$
 $T' \rightarrow *T | \epsilon$

table	int	*	+	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'			$E' \rightarrow +E$		$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow int T'$			$T \rightarrow (E)$		
T'		$T' \rightarrow *T$	$T' \rightarrow \epsilon$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$

Derivation

- Consider a CFG grammar G

$S \rightarrow AB$

$A \rightarrow aC$

$B \rightarrow bD$

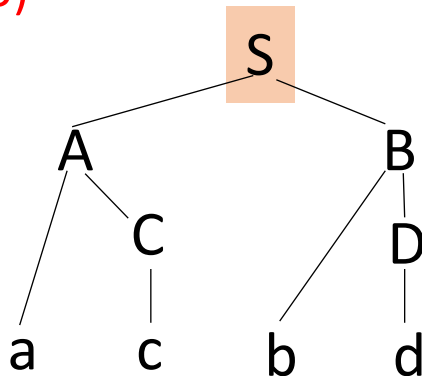
$D \rightarrow d$

$C \rightarrow c$

- This language has only one sentence: $L(G) = \{acbd\}$

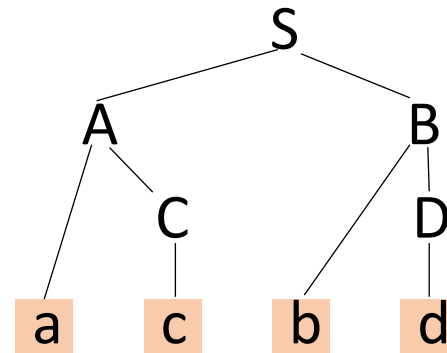
Top-down (leftmost derivation)

$S \Rightarrow AB$ (1)
 $\Rightarrow aCB$ (2)
 $\Rightarrow acB$ (3)
 $\Rightarrow acbD$ (4)
 $\Rightarrow acbd$ (5)



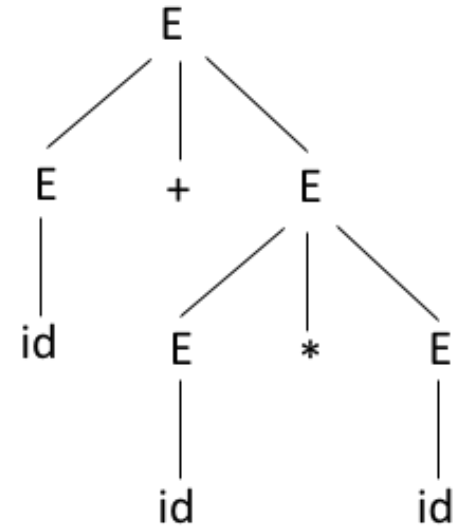
Bottom-up (reverse of rightmost derivation)

$S \Rightarrow AB$ (5)
 $\Rightarrow AbD$ (4)
 $\Rightarrow Abd$ (3)
 $\Rightarrow aCbd$ (2)
 $\Rightarrow acbd$ (1)



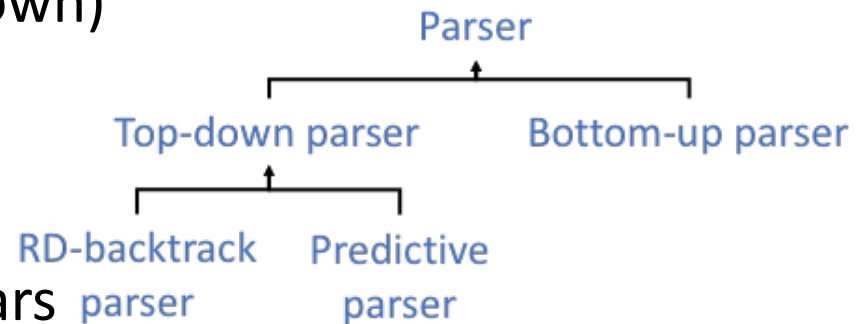
Parser Types[分析器类型]

- **Grammar** is used to derive string or construct **parser**
- Most compilers use either **top-down** or **bottom-up** parsers
- Top-down parsing[自顶向下分析]
 - Starts from root and expands into leaves
 - Tries to **expand start symbol to input string**
 - Finds leftmost derivation[最左推导]
 - In each step
 - Which non-terminal to replace?[哪个符号?]
 - Which production of the non-terminal to use?[哪个规则?]
 - Parser code structure closely mimics grammar
 - Amenable to implementation by hand
 - Automated tools exist to convert to code (e.g. ANTLR)



Bottom-up Parsing[自底向上]

- Begins at leaves and works to the top[叶子到根]
 - Bottom-up: **reduces**[归约] input string to start symbol
 - In the opposite direction from top-down
 - Top-down: expands start symbol to input string
 - In reverse order of rightmost derivation (In effect, builds tree from left to right, just like top-down)



- More powerful than top down
 - Don't need left factored grammars
 - Can handle left recursion
 - Can express a larger set of languages
 - And just as efficient



Apply LL(1) Parsing to Grammar[应用]

- Consider the grammar

$$E \rightarrow T+E \mid T$$

$$T \rightarrow \text{int} * T \mid \text{int} \mid (E)$$

- Left recursion? **NO!**
- Left factoring? **YES.** $E \rightarrow T+E \mid T$, $T \rightarrow \text{int} * T \mid \text{int}$

- After rewriting grammar, we have

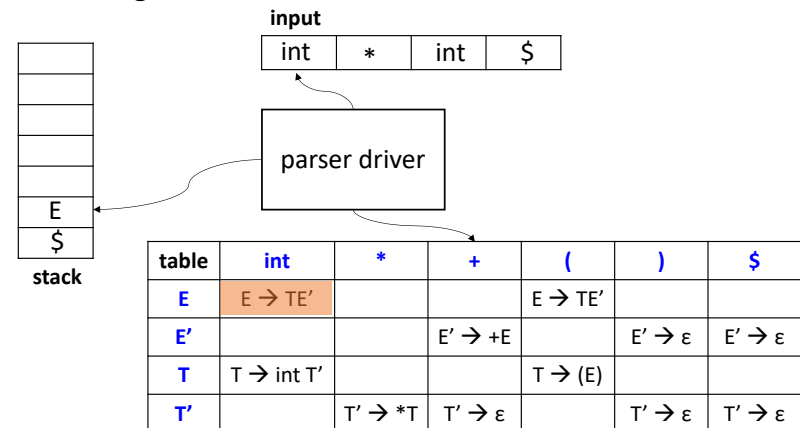
$$E \rightarrow TE'$$

$$E' \rightarrow +E \mid \epsilon$$

$$T \rightarrow \text{int} T' \mid (E)$$

$$T' \rightarrow *T \mid \epsilon$$

- To recognize “int * int”



Recognize Sequence[解析过程]

Matched	Stack (unmatched)	Input	Action
	E \$	int * int \$	$E \rightarrow TE'$
	T E' \$	int * int \$	$T \rightarrow \text{int } T'$
int	int T' E' \$	int * int \$	match
int	T' E' \$	* int \$	$T' \rightarrow *T$
int	* T E' \$	* int \$	match
int *	T E' \$	int \$	$T \rightarrow \text{int } T'$
int *	int T' E' \$	int \$	match
int * int	T' E' \$	\$	$T' \rightarrow \epsilon$
int * int	E' \$	\$	$E' \rightarrow \epsilon$
int * int	\$	\$	Halt-accept

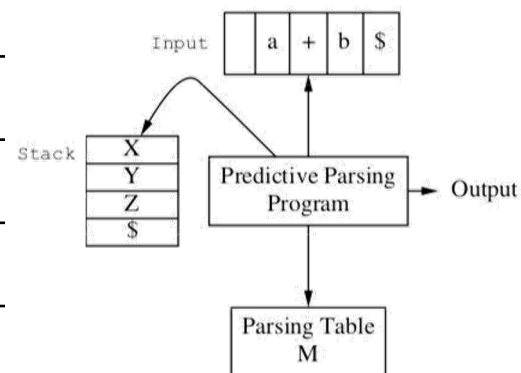
$E \rightarrow TE'$

$E' \rightarrow +E \mid \epsilon$

$T \rightarrow \text{int}T' \mid (E)$

$T' \rightarrow *T \mid \epsilon$

Input: int * int



- 'Matched + Stack' constructs the sentential form[句型]
- Actions correspond to productions in leftmost derivation

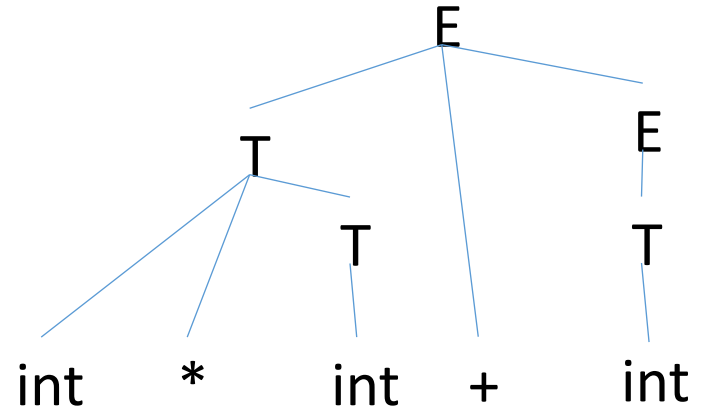
Example

- Grammar

$E \rightarrow T + E \mid T$

$T \rightarrow \text{int} * T \mid \text{int} \mid (E)$

- String: $\text{int} * \text{int} + \text{int}$



- The rightmost derivation of the parse tree

– $E \Rightarrow T + E \Rightarrow T + T \Rightarrow T + \text{int} \Rightarrow \text{int} * T + \text{int} \Rightarrow \text{int} * \text{int} + \text{int}$

- To recognize the string via bottom-up parsing

– $\text{int} * \text{int} + \text{int} \Rightarrow \text{int} * T + \text{int} \Rightarrow T + \text{int} \Rightarrow T + T \Rightarrow T + E \Rightarrow E$

Example (cont.)

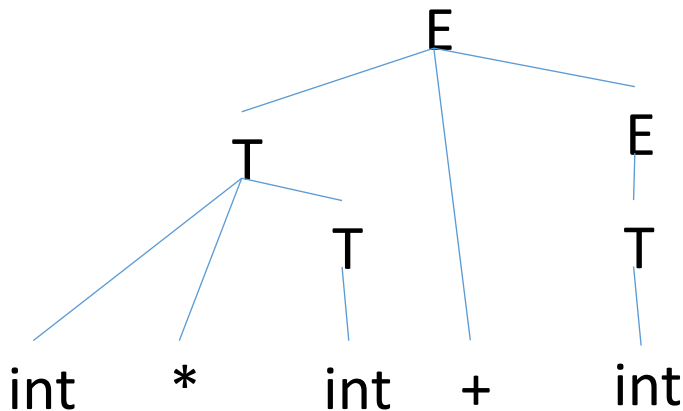
- Grammar

$$E \rightarrow T + E \mid T$$

$$T \rightarrow \text{int} * T \mid \text{int} \mid (E)$$

- String

int * int + int



Step	Operation
# int * int + int	Shift
int # * int + int	Shift
int * # int + int	Shift
int * int # + int	Reduce $T \rightarrow \text{int}$
int * T # + int	Reduce $T \rightarrow \text{int} * T$
T # + int	Shift
T + # int	Shift
T + int #	Reduce $T \rightarrow \text{int}$
T + T #	Reduce $E \rightarrow T$
T + E #	Reduce $E \rightarrow T + E$
E #	



Top-down vs. Bottom-up

Matched	Stack (unmatched)	Input	Action
	E \$	int * int \$	$E \rightarrow TE'$
	T E' \$	int * int \$	$T \rightarrow \text{int } T'$
int	int T' E' \$	int * int \$	match
int	T' E' \$	* int \$	$T' \rightarrow *T$
int	* T E' \$	* int \$	match
int *	T E' \$	int \$	$T \rightarrow \text{int } T'$
int *	int T' E' \$	int \$	match
int * int	T' E' \$	\$	$T' \rightarrow \epsilon$
int * int	E' \$	\$	$E' \rightarrow \epsilon$
int * int	\$	\$	Halt-accept

Top-down

Step	Operation
# int * int + int	Shift
int # * int + int	Shift
int * # int + int	Shift
int * int # + int	Reduce $T \rightarrow \text{int}$
int * T # + int	Reduce $T \rightarrow \text{int} * T$
T # + int	Shift
T + # int	Shift
T + int #	Reduce $T \rightarrow \text{int}$
T + T #	Reduce $E \rightarrow T$
T + E #	Reduce $E \rightarrow T + E$
E #	

Bottom-up

Bottom-up: Overview

- An important fact:
 - Let $\alpha\beta\omega$ be a step of a bottom-up parse[bottom-up解析过程中的一步]
 - Assume the next reduction is by $X \rightarrow \beta$ [下一步, 把 β 归约到 X]
 - Then ω is a string of terminals[i.e., ω 一定是句子]
- Why? $\alpha X\omega \rightarrow \alpha\beta\omega$ is a step in a rightmost derivation
- Idea: split string into two substrings
 - Right substring is as yet unexamined by parsing (a string of terminals)[右侧尚未被解析, i.e., 最右推导中已被完全展开]
 - Left substring has terminals and non-terminals[左侧已有解析, i.e., 最右推导中尚未被完全展开]
- The dividing point is marked by a #
 - The # is not part of the string[仅作为标示]
 - Initially, all input is unexamined $\#x_1x_2 \dots x_n$ [输入尚未有任何解析]

Bottom-up: Shift-Reduce[移入-归约]

- Bottom-up parsing is also known as **Shift-Reduce** parsing
 - Involves two types of operations: shift and reduce
 - Recall: expand and compare operations for top-down parsing
- **Shift**[移入]: move # one place to the right
 - Shifts a terminal to the left string[向左侧移入终结符，推进解析]
 $ABC\#xyz \Rightarrow ABCx\#yz$
- **Reduce**[归约]: apply an inverse production at the right end of the left string[左侧串的右端进行归约]
 - If $E \rightarrow Cx$ is a production, then
 $ABCx\#yz \Rightarrow ABEx\#yz$

Stack[栈]

- Left string can be stored into a **stack**
 - Top of the stack is the **#**[左右串的分界]
- **Shift** pushes a terminal on the stack
- **Reduce** does the following:
 - pops zero or more symbols off the stack
 - production *rhs*[pop出了产生式RHS]
 - pushes a non-terminal on the stack
 - production *lhs*[push进了产生式LHS]
 - just reverts production ($LHS \leftarrow RHS$)[产生式逆向使用]

Step
int * int + int
int # * int + int
int * # int + int
int * int # + int
int * T # + int
T # + int
T + # int
T + int #
T + T #
T + E #
E #

Key Issue[一个关键问题]

- How to decide when to shift or reduce?

- Example grammar:

$E \rightarrow T + E \mid T$

$T \rightarrow \text{int} * T \mid \text{int} \mid (E)$

#int * int + int	Shift
int# * int + int	Shift
int * #int + int	Shift
... ..	✓

#int * int + int	Shift
int# * int + int	Reduce $T \rightarrow \text{int}$
T # * int + int	Shift
... ..	✗

- Consider the step $\text{int} \# * \text{int} + \text{int}$

- We could reduce by $T \rightarrow \text{int}$ giving $T \# * \text{int} + \text{int}$

- A fatal mistake: no way to reduce to the start symbol E

- **Intuition:** want to reduce only if the result can still be reduced to the start symbol[必须在对的方向上]



The Example

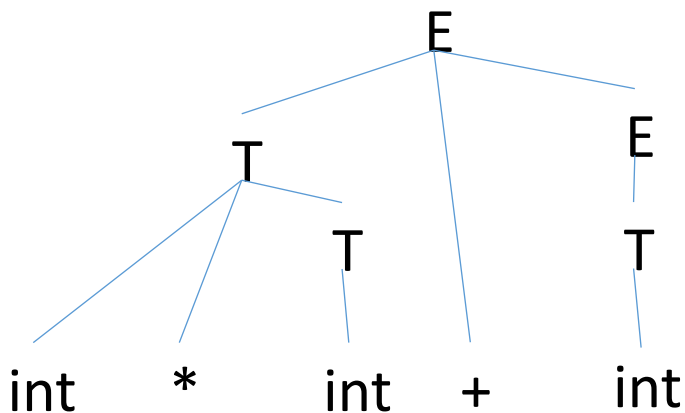
- Grammar

$$E \rightarrow T + E \mid T$$

$$T \rightarrow \text{int} * T \mid \text{int} \mid (E)$$

- String

int * int + int



Step	Operation
# int * int + int	Shift
int # * int + int	Shift
int * # int + int	Shift
int * int # + int	Reduce $T \rightarrow \text{int}$
int * T # + int	Reduce $T \rightarrow \text{int} * T$
T # + int	Shift
T + # int	Shift
T + int #	Reduce $T \rightarrow \text{int}$
T + T #	Reduce $E \rightarrow T$
T + E #	Reduce $E \rightarrow T + E$
E #	

Why not just $E \rightarrow T$?
 $(\text{int} * T \Rightarrow \text{int} * E)$

Handle[句柄]

- A **handle** of a sentential form is a substring α such that:
 - α matches the RHS of a production $A \rightarrow \alpha$; and[能匹配上规则]
 - replacing α by the LHS A represents a step in the reverse of a rightmost derivation of S [且能推进解析]
- Definition: let $\alpha\beta\omega$ be a sentential form where:
 - α, β is a string of terminals and non-terminals (yet to be derived)
 - ω is a string of terminals (already derived)
 - Then β is a **handle** of $\alpha\beta\omega$ if: $S \Rightarrow_{rm}^* \alpha X \omega \Rightarrow \alpha\beta\omega$ by a rightmost/rm derivation (apply rule $X \rightarrow \beta$)
- 🖐️ We only want to reduce at handles, and there is exactly one handle per sentential form
 - But **where to find it?**

Some Concepts[一些概念]

- A **right-sentential form**[最右句型] is a sentential form that occurs in the rightmost derivation of some sentence
- A **phrase**[短语] is a subsequence of a sentential form that is eventually “reduced” to a single non-terminal[能归约到单个非终结符]
 - β is a phrase of the right sentential form γ iff $S \Rightarrow_{rm}^* \gamma = \alpha_1 A \alpha_2 \Rightarrow^+ \alpha_1 \beta \alpha_2$
 - a string consisting of all of the leaves of the partial parse tree that is rooted at one particular internal node of the whole parse tree[一个句型的语法树中任一子树叶结点所组成的符号串都是该句型的短语]
- A **simple phrase**[直接短语] is a phrase that is reduced in a single step
 - β is a simple phrase of the right sentential form γ iff $S \Rightarrow_{rm}^* \gamma = \alpha_1 A \alpha_2 \Rightarrow \alpha_1 \beta \alpha_2$
- The **handle** is the leftmost simple phrase[最左直接短语]

Handle: Example

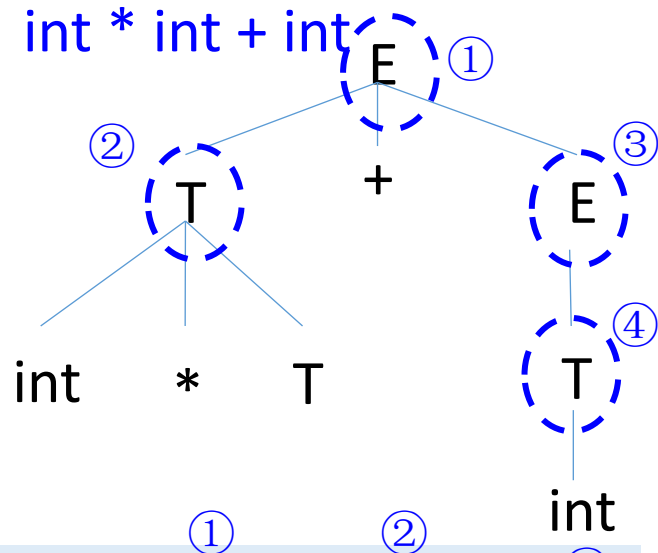
- Grammar

$$E \rightarrow T + E \mid T$$

$$T \rightarrow \text{int} * T \mid \text{int} \mid (E)$$

- String

int * int + int



Phrase: int * T + int, int * T, int

Simple phrase: int * T, int

Handle: int * T

Step	Operation
# int * int + int	Shift
int # * int + int	Shift
int * # int + int	Shift
int * int # + int	Reduce $T \rightarrow \text{int}$
int * T # + int	Reduce $T \rightarrow \text{int} * T$
T # + int	Shift
T + # int	Shift
T + int #	Reduce $T \rightarrow \text{int}$
T + T #	Reduce $E \rightarrow T$
T + E #	Reduce $E \rightarrow T + E$
E #	

One More Example

- Grammar

$E \rightarrow E+T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid \text{id}$

- Sentential form:

– $T + T * F + \text{id}$

- Phrase:

– $T + T * F + \text{id}$ ①

– $T + T * F$ ②

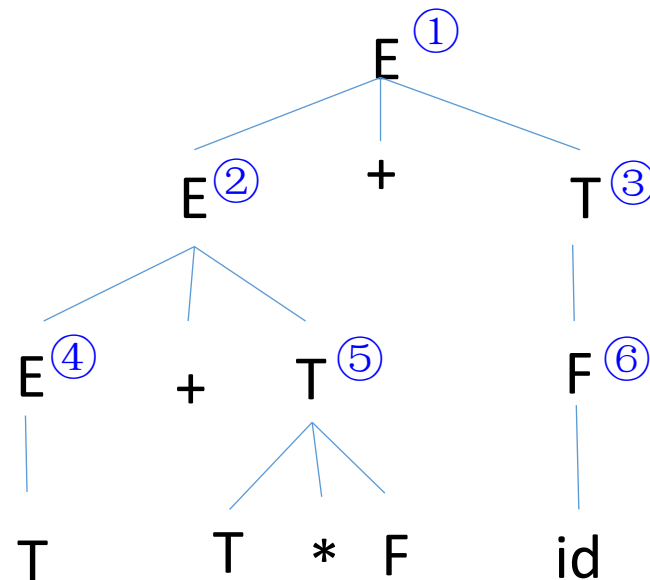
– T ④

– $T * F$ ⑤

– id ③ ⑥

Handle

Simple phrase



Handle Always Occurs at Stack Top

- Why can't a handle occur on right side of #?[不在栈外]
 - It can
 - But handle will eventually be shifted in, placing it at top of stack
 - In $\text{int} * \# \text{int} + \text{int} \Rightarrow \text{int} * \text{int} \# + \text{int}$, int is eventually shifted to the top (i.e., left to $\#$)

- Why can't a handle occur on left side of $\#$, i.e., in middle of the stack?[不在栈‘中’]

$\text{int} * \# \text{int} + \text{int}$	Shift
$\text{int} * \text{int} \# + \text{int}$	Reduce $T \rightarrow \text{int}$
$\text{int} * T \# + \text{int}$	Reduce $T \rightarrow \text{int} * T$

- Can $\text{int} * \text{int} + \# \text{int}$ occur? NOPE.
 - Means parser shifted when it could have reduced when the handle was on top[本应归约却移入]
 - If eagerly reduces when handle is at top of stack, never occurs
- 🖐️ Makes life easier for parser (need only access top of stack)

Viable Prefix[活前缀]

- In shift-reduce parsing, the stack contents are always a **viable prefix**[活前缀/可动前缀]
 - A prefix of some right-sentential form that ends no further right than the end of the handle of that right-sentential form
 - A viable prefix has a handle at its rightmost end
 - Stack content is always a viable prefix, guaranteeing the shift / reduce is on the right track[活前缀说明移进归约是正确的]
- 定义：一个可行前缀是一个最右句型的前缀，并且它没有越过该最右句型的最右句柄的右端
 - 举例： $S \Rightarrow bBa \Rightarrow bbAa$ ，这里句柄是 bA ，因此可行前缀包括 bA 的所有前缀（包括 b, bb, bbA ），但不能是 $bbAa$ （因为越过了句柄）

... $\Rightarrow T + \text{int} \Rightarrow \text{int} * T + \text{int}$

Handle: $\text{int} * T$

Viable prefix: $\text{int}, \text{int} *, \text{int} * T$

$\text{int} * \# \text{int} + \text{int}$	Shift
$\text{int} * \text{int} \# + \text{int}$	Reduce $T \rightarrow \text{int}$
$\text{int} * T \# + \text{int}$	Reduce $T \rightarrow \text{int} * T$
$T \# + \text{int}$	Shift